

Lab Manual: Flask ETL App with Celery and Redis Using Docker Compose

Objective

By the end of this lab, you will:

- Build a Flask API with an endpoint to trigger ETL jobs.
- Use **Celery** for background task execution.
- Use **Redis** as the broker and result backend.
- Run everything together with **Docker Compose**.

Prerequisites

- Docker installed.
- Docker Compose installed.
- Basic knowledge of Flask & Celery.

Step 1: Project Setup

Create a folder:

```
mkdir flask_etl_docker
cd flask_etl_docker
```

Inside, create these files:

```
flask_etl_docker/
|— app.py
|— tasks.py
|— requirements.txt
```

- |— Dockerfile
- |— Dockerfile.worker
- |— docker-compose.yml

Step 2: Define Dependencies

requirements.txt

```
flask
celery
redis
```

This ensures Flask, Celery, and Redis client are installed in the containers.

Step 3: Flask App (API)

app.py

```
from flask import Flask, request, jsonify
from tasks import run_etl

app = Flask(__name__)

@app.route("/run-etl", methods=["POST"])
def trigger_etl():
    param = request.json.get("param", "default")
    task = run_etl.delay(param) # Enqueue Celery task
    return jsonify({"task_id": task.id}), 202

if __name__ == "__main__":
    app.run(host="0.0.0.0", port=5000)
```

Explanation

- `/run-etl` → accepts POST requests with a JSON payload.
- `run_etl.delay()` → queues a background ETL task via Celery.
- Returns `task_id` with HTTP **202 Accepted**.

Step 4: Celery Task

`tasks.py`

```
from celery import Celery
import time

# Configure Celery to use Redis
celery_app = Celery(
    "etl",
    broker="redis://redis:6379/0",
    backend="redis://redis:6379/0"
)

@celery_app.task
def run_etl(param):
    print(f"Starting ETL with param: {param}")
    time.sleep(5) # Simulate ETL work
    print("ETL complete")
    return f"Processed with param: {param}"
```

Explanation

- `Celery(... broker=..., backend=...)` → uses Redis for both job queue and result storage.
- `@celery_app.task` → marks `run_etl` as a Celery task.
- `time.sleep(5)` simulates a long-running ETL job.

Step 5: Dockerfile for Flask Web

Dockerfile

```
FROM python:3.10-slim

WORKDIR /app

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY . .

CMD ["python", "app.py"]
```

Step 6: Dockerfile for Celery Worker

Dockerfile.worker

```
FROM python:3.10-slim

WORKDIR /app

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY . .

CMD ["celery", "-A", "tasks.celery_app", "worker", "--loglevel=info"]
```

Step 7: Docker Compose File

docker-compose.yml

```
version: "3.9"
```

```
services:
```

```
  web:
```

```
    build:
```

```
      context: .
```

```
      dockerfile: Dockerfile
```

```
    ports:
```

```
      - "5000:5000"
```

```
    depends_on:
```

```
      - redis
```

```
  worker:
```

```
    build:
```

```
      context: .
```

```
      dockerfile: Dockerfile.worker
```

```
    depends_on:
```

```
      - redis
```

```
  redis:
```

```
    image: redis:7-alpine
```

```
    ports:
```

```
      - "6379:6379"
```

Explanation

- **web** → Flask API, mapped to host port 5000.
- **worker** → Celery worker that consumes jobs.
- **redis** → message broker + result backend.
- All services share a network, so **redis** is accessible via hostname **redis**.

Step 8: Run the Stack

Build and start all containers:

```
docker compose up --build
```

You'll see logs from Flask, Celery, and Redis.

Run in background:

```
docker compose up -d --build
```

Stop services:

```
docker compose down
```

Step 9: Test the Workflow

Trigger a job:

```
curl -X POST http://localhost:5000/run-etl \
  -H "Content-Type: application/json" \
  -d '{"param":"my_test"}'
```

Response:

```
{"task_id": "d8e2d1b4-7f3b-4df7-8f4c-1c9e8b7f9a1a"}
```

Check Celery worker logs:

```
Starting ETL with param: my_test
ETL complete
```

The job ran asynchronously!

Verification Checklist

- All required files created (app.py, tasks.py, requirements.txt, Dockerfile, Dockerfile.worker, docker-compose.yml).
- Containers built and started via `docker compose up --build`.
- `/run-etl` endpoint returns a task ID.
- Celery worker logs confirm ETL execution.

Summary

In this lab, you:

- Built a **Flask API** to trigger ETL jobs.
- Defined **Celery tasks** for background execution.
- Used **Redis** as the broker + result backend.
- Containerized everything and orchestrated with **Docker Compose**.

This setup is the foundation for **scalable data pipelines**:

- Add more **workers** to scale ETL processing.
- Use Celery monitoring tools like **Flower** for visibility.
- Replace the dummy ETL with **PySpark, DB writes, or S3 uploads**.